A comprehensive systems documentation of Synergetics operating system

Authorship, Ownership, and Contribution Policy

Original Conception

The concept of the **Synergetic Operating System (SOS)** is the original intellectual conception of:

Solomon Yaw Adeklo

- GitHub Profile: https://github.com/solomonyaw
- Project Repository: https://github.com/solomonyaw/Synergetics-Operating-System

📄 **Whitepaper**:
https://github.com/solomonyaw/Synergetics-Operating-System/blob/master/SYNERGETIC%20OPERATING%20SYSTEM%20-%20White%20Paper.pdf

📄 **Simulation Code**: https://github.com/solomonyaw/Synergetics-Operating-System/blob/master/synergetic_scheduler_rust_simulation.rs
This work represents a novel systems design philosophy that integrates principles of Synergetics, topology, and emergent computation into operating system architecture.

Development Background

Solomon Yaw Adeklo:

- Does **not have prior system-level programming experience**
- Has experience primarily in **application-level programming**
- Conceived the architecture and guiding principles independently
- Utilized **generative AI tools** to:
  - Develop technical documentation
  - Explore system design concepts
  - Build early-stage simulations

This distinction is important:

The originality lies in the **conceptual framework and system design philosophy**, not in prior traditional OS engineering experience.

Open Source Status

This project is **open source**, intended to:

- Encourage research and experimentation
- Enable collaborative development
- Advance new paradigms in operating system design

Usage and Intellectual Respect Clause

While this project is open source, the following expectations apply:

- Individuals, organizations, and institutions are **expected to acknowledge** the original conception by Solomon Yaw Adeklo when using or building upon this work.

- Large organizations (including major technology companies) are **strongly encouraged to seek engagement, collaboration, or approval** before implementing core ideas derived directly from this project.

This clause is not a legal restriction by itself unless paired with a formal license, but it establishes:

clear authorship, attribution expectations, and ethical use guidelines

---

Collaboration and Contributions

Contributors are welcome.

If you wish to collaborate, contribute, or support development:

📧 Contact: **sadeklo@st.vvu.edu.gh**

Areas of contribution include:

- Systems programming (Rust, kernel development)
- Scheduler design and optimization
- Distributed systems
- Visualization and simulation
- Documentation and research

Future Direction

This project is in an early-stage, research-driven phase and aims to evolve into:

- A functional prototype OS
- A new model for adaptive and emergent computing systems
- A collaborative open-source research initiative

Recommended Next Step (Important)

If you want stronger protection and clarity, you should attach a license such as:

- **Apache 2.0** (with attribution + patent protection)

- **GPLv3** (forces open-source reuse)
- Or a **custom license** if you want stricter control

📄 LICENSE

Synergetic Operating System License (SOSL-1.0)

Copyright (c) 2026

Solomon Yaw Adeklo

GitHub: https://github.com/solomonyaw
Repository: https://github.com/solomonyaw/Synergetics-Operating-System

Whitepaper: https://github.com/solomonyaw/Synergetics-Operating-System/blob/master/SYNERGETIC%20OPERATING%20SYSTEM%20-%20White%20Paper.pdf

---

1. Definitions

"The Work" refers to the Synergetic Operating System (SOS), including:

- Source code
- Documentation
- Architecture designs
- Simulations
- Derivative technical models based on the original conceptual framework

"Contributor" means any individual or entity contributing to the Work.

"Large Organization" means:

- Any corporation with more than 100 employees, OR
- Any entity generating more than USD $1,000,000 annual revenue, OR
- Any government agency or defense contractor

---

2. Ownership and Attribution

The Synergetic Operating System is the **original intellectual conception of Solomon Yaw Adeklo**.

All copies, modifications, or derivative works must:

- Retain clear attribution to the original author
- Include this license in all distributions
- Preserve references to the original repository

---

3. Permission to Use

Permission is hereby granted, free of charge, to any person or organization to:

- Use the Work for research and educational purposes
- Modify and experiment with the Work
- Contribute improvements back to the project
- Distribute non-commercial copies with attribution

---

4. Restrictions (Critical Clause)

4.1 Corporate and Institutional Use Restriction

No Large Organization may:

- Deploy the Work in production systems
- Integrate the Work into commercial or proprietary software
- Use the Work as part of internal infrastructure
- Patent or claim ownership over derivative implementations

WITHOUT explicit written permission from:

Solomon Yaw Adeklo

4.2 Derivative Works

Any derivative work must:

- Remain open and accessible under the same license terms
- Not be converted into proprietary software

- Not remove attribution to the original author

---

5. Open Source Collaboration Clause

This project is **open for collaboration**.

Contributors are encouraged to:

- Extend system architecture
- Improve scheduler models
- Develop kernel-level implementations
- Enhance simulation and visualization systems

All contributions automatically become part of the Work under this license.

---

6. AI-Assisted Development Disclosure

The original author acknowledges that:

- Generative AI tools were used in the development of documentation and simulation artifacts
- The **core conceptual architecture remains original intellectual work**
- Implementation guidance may evolve through collaborative development

---

7. No Warranty

THE WORK IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND.
THE AUTHOR IS NOT LIABLE FOR ANY DAMAGES ARISING FROM ITS USE.

---

8. Contact for Authorization

For collaboration, licensing requests, or corporate approval:

sadeklo@st.vvu.edu.gh

---

9. Termination

Any violation of this license automatically terminates usage rights.

---

10. Final Statement

This license exists to protect:

- The originality of the Synergetic Operating System concept
- The integrity of its open research direction
- The ethical use of its architectural framework

Synergetic Operating Systems:

Redesigning Computing Through Adaptive, Geometric, and Emergent Architectures**

Abstract

This document proposes a novel paradigm for operating system (OS) design inspired by the principles of **Synergetics** and **ephemeralization**, as developed by Buckminster Fuller. Traditional operating systems rely on hierarchical, reductionist architectures grounded in deterministic logic and linear resource management. In contrast, a Synergetic Operating System (SOS) reimagines computation as a **holistic, relational, and geometry-driven system**, enabling adaptive behavior, resource efficiency, and emergent optimization.

The objective is not to replace mathematical computation, but to **elevate system design philosophy**, allowing operating systems to evolve toward self-organizing, energy-aware, and topology-native computational environments.

---

1. Introduction

Modern operating systems such as Windows, Linux, and macOS are built upon:

- Hierarchical abstractions (kernel → processes → threads)

- Static resource allocation models
- Deterministic scheduling algorithms

While effective, these approaches are constrained by:

- Legacy design assumptions
- Inefficient resource utilization
- Limited adaptability to dynamic workloads

Synergetics introduces an alternative worldview:

Systems are best understood not by their parts, but by the **relationships and patterns** that emerge from their interactions.

This document explores how that worldview can inform the design of a next-generation operating system.

---

2. Theoretical Foundations

2.1 Synergetics

Synergetics studies:

- Whole-system behavior
- Geometric relationships
- Emergent properties not predictable from individual components

Core implications for OS design:

- Replace component-centric thinking with **relationship-centric modeling**
- Treat computation as a **network of interactions**, not isolated processes

---

2.2 Ephemeralization

Ephemeralization is defined as:

“Doing more with less.”

Applied to computing:

- Minimize computational overhead

- Maximize output per unit of energy, memory, and processing time
- Eliminate persistent structures unless absolutely necessary

---

2.3 From Hierarchy to Topology

Traditional OS:

- Tree structures (process trees, file systems)

Synergetic OS:

- **Graph or field structures**
- Nodes represent processes, memory units, or resources
- Edges represent dynamic relationships

---

3. Core Design Principles of a Synergetic Operating System

3.1 Holistic System Architecture

- The OS is modeled as a **continuous dynamic system**
- No rigid separation between kernel and user space
- Boundaries are adaptive and context-dependent

3.2 Geometric Computation Model

- Processes and resources exist in a **multi-dimensional state space**
- Scheduling becomes a problem of **spatial optimization**
- Data flow is represented as vectors within a system geometry

This enables:

- Efficient routing of computation
- Reduced latency through proximity-based execution

3.3 Resource Field Abstraction

Instead of discrete resources:

- CPU
- Memory
- Storage

The OS defines a unified:

Resource Field

Characteristics:

- Resources are dynamically allocated as needed
- No static ownership of resources
- Continuous redistribution based on system demand

3.4 Emergent Optimization

- The system does not rely solely on predefined rules
- Optimization emerges from:
    - Local interactions
    - Feedback loops
    - System-wide constraints

Examples:

- Processes reorganize themselves for efficiency
- Load balancing occurs naturally through system dynamics

3.5 Ephemeral Process Model

- Processes are not permanent entities
- They:
    - Emerge when needed
    - Dissolve when idle

Benefits:

- Reduced memory footprint
- Elimination of zombie processes
- Near-zero idle resource consumption

---

4. Architectural Components

4.1 Topological Kernel

- Replaces traditional monolithic or microkernel designs
- Core responsibility:
  - Maintain system relationships
  - Manage dynamic topology

Key features:

- Graph-based process management
- Relationship-driven scheduling

4.2 Adaptive Scheduler

- Moves beyond time-slicing
- Uses:
  - Proximity (data/process closeness)
  - Load gradients
  - Energy efficiency metrics

Outcome:

- Optimal task placement across the system

4.3 Dynamic Memory Fabric

- Memory is not statically allocated
- Functions as a **fluid substrate**

Capabilities:

- Real-time memory redistribution
- Data locality optimization
- Reduced fragmentation

4.4 Feedback and Learning Layer

- Continuous monitoring of system state
- Incorporates:
    - Machine learning models (optional)
    - Heuristic adaptation

Purpose:

- Enable self-optimization over time

---

5. Advantages Over Traditional Systems

5.1 Efficiency

- Reduced overhead through ephemeralization
- Better utilization of hardware resources

5.2 Scalability

- Naturally suited for:
    - Distributed systems
    - Cloud environments
    - Edge computing

5.3 Adaptability

- Responds dynamically to workload changes
- Eliminates rigid scheduling bottlenecks

5.4 Resilience

- No single point of failure
- System reorganizes under stress

---

6. Technical Challenges

6.1 Determinism vs Emergence

- Ensuring predictable behavior in an emergent system

6.2 Debugging Complexity

- Harder to trace issues in dynamic topologies

6.3 Hardware Constraints

- Current CPUs are not optimized for geometric computation models

---

6.4 Compatibility

- Existing software ecosystems rely on traditional OS APIs

---

7. Implementation Roadmap

Phase 1: Simulation Layer

- Build a user-space simulation:
  - Graph-based process model
  - Experimental scheduler

Phase 2: Kernel Prototype

- Develop a minimal kernel:
  - Language: Rust or C
  - Focus:
    - Process topology
    - Resource field abstraction

Phase 3: Hybrid Integration

- Integrate Synergetic components into existing OS (e.g., Linux)

- Test:
    - Adaptive scheduling
    - Dynamic memory allocation

Phase 4: Full Synergetic OS

- Independent operating system
- Designed from first principles

---

8. Future Implications

A Synergetic OS could enable:

- Autonomous computing environments
- Ultra-efficient cloud infrastructure
- AI-native operating systems
- New paradigms in human-computer interaction

---

9. Conclusion

The application of Synergetics to operating system design represents a shift from:

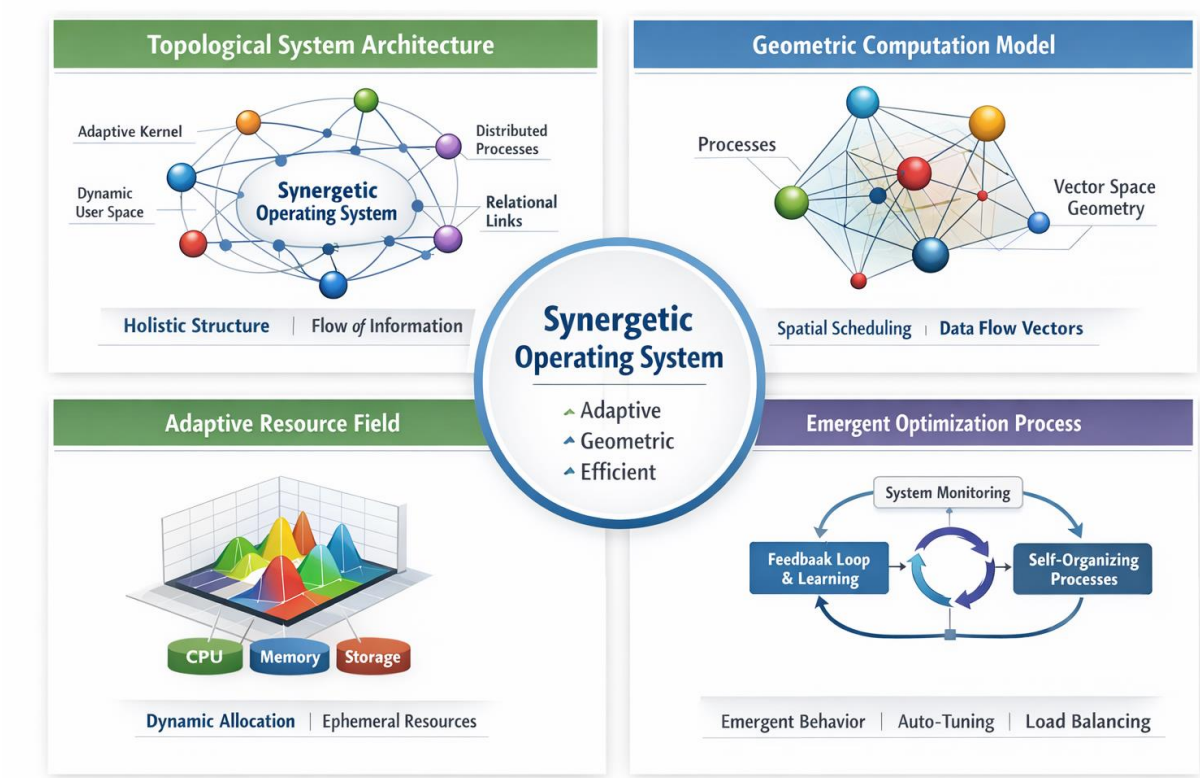
- Static → Dynamic
- Hierarchical → Topological
- Deterministic → Emergent

While significant technical challenges remain, the integration of **geometric thinking, adaptive systems, and ephemeral efficiency** offers a credible pathway toward the next evolution of computing.

The central thesis stands:

Operating systems can be redesigned as **living systems of relationships**, where efficiency, intelligence, and resilience emerge naturally from the structure of the whole.

Below is the architecture diagram of the Synergetics Operating System

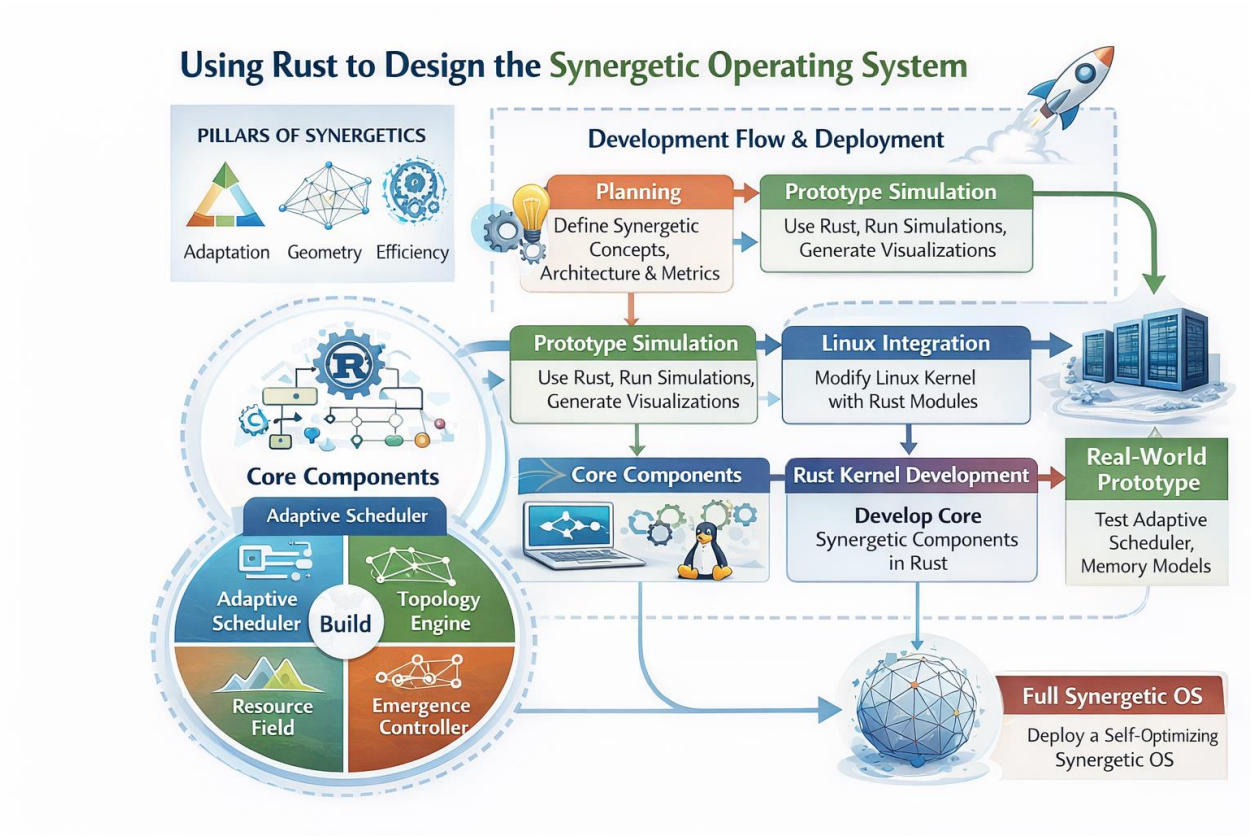


Below is a **developer-oriented technical document** that shows how to use **Rust** to build your Synergetic Operating System (SOS), while also integrating with an **existing kernel (Linux)** for faster validation.

Building a Synergetic Operating System with Rust

A Practical Guide to Topological, Adaptive, and Emergent OS Design

---



1. Overview

This document outlines how to implement a **Synergetic Operating System (SOS)** using **Rust**, guided by the principles of Buckminster Fuller's **Synergetics** and ephemeralization.

Two parallel development paths are defined:

1. **Greenfield (Custom Rust Kernel)** → long-term vision
2. **Brownfield (Linux-based Integration)** → short-term validation

---

2. Why Rust for a Synergetic OS

2.1 Core Advantages

Rust provides features that map directly to your system philosophy:

Synergetic Concept	Rust Feature
Resource relationships	Ownership & borrowing
Ephemeral systems	RAII (automatic cleanup)
Emergent concurrency	Fearless concurrency
System integrity	Memory safety (no GC)

2.2 Key Rust Capabilities for OS Development

- `no_std` environment (no standard library)
- Direct hardware interaction
- Zero-cost abstractions
- Deterministic memory behavior

---

3. Development Paths

PATH A: Building a Custom Rust Kernel

3.1 Environment Setup

Install Rust nightly

```
rustup install nightly
rustup default nightly
rustup component add rust-src
```

Add target for bare metal

```
rustup target add x86_64-unknown-none
```

3.2 Minimal Kernel Structure

```
synergetic_os/
├── Cargo.toml
├── src/
│   ├── main.rs
│   ├── topology/
│   ├── scheduler/
│   ├── resource_field/
│   └── memory/
```

3.3 Bare-Metal Entry Point

```rust
#![no_std]
#![no_main]

use core::panic::PanicInfo;

#[no_mangle]
pub extern "C" fn _start() -> ! {
    loop {}
}

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}
```

3.4 Designing the Topological Core

This is where your innovation begins.

Process as a Node

```rust
pub struct Node {
    pub id: usize,
    pub connections: Vec<usize>,
    pub resource_weight: u32,
}
```

System as a Graph

```rust
pub struct Topology {
    pub nodes: Vec<Node>,
}

impl Topology {
    pub fn add_connection(&mut self, a: usize, b: usize) {
        self.nodes[a].connections.push(b);
        self.nodes[b].connections.push(a);
    }
```

```
}
```

3.5 Synergetic Scheduler

Replace time-slicing with **relationship-based scheduling**:

```
impl Topology {
    pub fn select_next(&self) -> Option<&Node> {
        self.nodes
            .iter()
            .max_by_key(|n| n.resource_weight + n.connections.len() as u32)
    }
}
```

Concept:

- Nodes with more relationships = higher priority
- Resource demand influences scheduling

3.6 Resource Field Engine

Unify CPU, memory, and IO:

```
pub struct ResourceField {
    pub total_energy: u64,
}

impl ResourceField {
    pub fn allocate(&mut self, amount: u64) -> bool {
        if self.total_energy >= amount {
            self.total_energy -= amount;
            true
        } else {
            false
        }
    }
}
```

3.7 Memory Model (Ephemeralization)

- No persistent allocation unless necessary
- Automatic deallocation via scope

Example:

```
fn ephemeral_process() {
    let data = [0u8; 1024]; // freed automatically
}
```

3.8 Key Crates for OS Development

- `bootloader`
- `x86_64`
- `spin`
- `linked_list_allocator`

PATH B: Using an Existing Kernel (Linux Integration)

---

4. Why Use Linux First

Linux kernel provides:

- Hardware abstraction
- Process management
- Memory handling

This allows you to focus on:

your Synergetic layer instead of low-level infrastructure

---

5. Integration Strategies

5.1 User-Space Prototype (FASTEST START)

Build your Synergetic system as a **Rust application**:

Architecture:

```
Linux Kernel
   ↓
Rust Synergetic Engine
   ↓
Processes (mapped as nodes)
```

Example (Graph-based scheduler simulation)

```
use std::collections::HashMap;

struct Node {
    id: usize,
    connections: Vec<usize>,
    weight: u32,
}

struct System {
    nodes: HashMap<usize, Node>,
}

impl System {
    fn optimize(&self) -> Option<&Node> {
        self.nodes.values().max_by_key(|n| n.weight + n.connections.len() as
u32)
    }
}
```

5.2 eBPF / Kernel Hooks (Intermediate)

Use Linux extensibility:

- eBPF programs
- Scheduler hooks

Goal:

- Inject **adaptive scheduling logic**

5.3 Rust in the Linux Kernel (Advanced)

Linux now supports Rust modules.

You can:

- Write kernel modules in Rust

- Replace or augment subsystems

Example areas:

- Scheduler extensions
- Memory management policies

---

6. Mapping Synergetics to Linux

Synergetic Concept	Linux Implementation
Topology	Process graph in user space
Resource Field	cgroups + scheduler
Emergence	Feedback loops + monitoring
Ephemeral processes	Containers / short-lived processes

---

7. Hybrid Architecture (Recommended)

```
                +----------------------+
                |  Synergetic Engine   |
                |  (Rust, Topology)    |
                +----------+-----------+
                           |
        +------------------+------------------+
        |                                     |
+-------v--------+                   +--------v--------+
| Linux Kernel   |                   | Hardware        |
| (Base System)  |                   | CPU, Memory, IO |
+----------------+                   +-----------------+
```

---

8. Development Roadmap

Phase 1: Simulation

- Rust user-space graph system
- Test emergent scheduling

Phase 2: Linux Integration

- Map real processes into topology
- Implement adaptive scheduler logic

Phase 3: Kernel Module

- Introduce Rust kernel extensions
- Hook into scheduler

Phase 4: Custom Kernel

- Replace Linux entirely
- Full Synergetic OS

---

9. Critical Design Rules

1. Everything is a Relationship

- No isolated processes
- Only nodes and connections

---

2. Minimize Persistence

- Use ephemeral allocation
- Destroy unused structures quickly

---

3. Optimize Globally, Not Locally

- Avoid greedy scheduling
- Use system-wide awareness

---

4. Feedback is Mandatory

- System must observe itself

- Continuous adaptation

---

10. Final Perspective

Rust does not just help you build an OS—it enforces a discipline that aligns with Synergetics:

- Ownership → resource awareness
- Lifetimes → ephemeral existence
- Types → structural integrity

Using an existing kernel like Linux allows you to:

- Validate ideas quickly
- Avoid low-level bottlenecks
- Focus on your **core innovation: topology + emergence**

Below is a **developer-grade specification** for a **Topology-Based Scheduler (TBS)** tailored to your Synergetic OS. It replaces time-slicing with **relationship-aware, field-optimized scheduling** and is implementable both in user space (prototype) and kernel space (Rust or Linux integration).

Topology-Based Scheduler (TBS)

A Synergetic, Graph-Driven Scheduling Algorithm

---

1. Conceptual Model

1.1 System Representation

The OS is modeled as a **dynamic weighted graph**:

- **Node** = execution unit (process, thread, task)
- **Edge** = relationship (IPC, shared memory, data flow, dependency)
- **Field** = global resource state (CPU, memory, IO unified)

$$
G = (V, E, W)
$$

Where:

- (V): set of nodes
- (E): set of edges
- (W): weights (resource demand, priority, latency sensitivity)

---

2. Core Scheduling Objective

Instead of fairness alone, TBS optimizes:

Global system coherence + resource efficiency + latency minimization

Formally:

$$
\text{Select } v_i \in V \text{ that maximizes } S(v_i)
$$

---

3. Scoring Function (Priority Metric)

Each node gets a **dynamic score**:

$$
S(v) = \alpha R(v) + \beta C(v) + \gamma L(v) + \delta E(v) - \epsilon D(v)
$$

Where:

- **(R(v))** → Resource urgency (CPU/memory need)
- **(C(v))** → Connectivity (graph centrality)
- **(L(v))** → Locality (data proximity)
- **(E(v))** → Emergence factor (system-wide benefit)
- **(D(v))** → Decay (recent CPU usage / fairness penalty)
- (α, β, γ, δ, ϵ) → tunable weights

---

4. Metric Definitions (Precise)

4.1 Resource Urgency (R(v))

$$
R(v) = \frac{\text{requested resources}}{\text{allocated resources} + 1}
$$

Interpretation:

- Starving processes rise in priority

4.2 Connectivity (C(v))

Use **degree or weighted centrality**:

$$
C(v) = \sum_{u \in N(v)} w(v,u)
$$

Meaning:

- Highly connected nodes = system-critical

4.3 Locality (L(v))

$$
L(v) = \frac{1}{\text{distance to data/resources}}
$$

Implementation:

- NUMA distance
- cache proximity
- memory region closeness

4.4 Emergence Factor (E(v))

This is your **Synergetic innovation**.

$$
E(v) = \sum (\text{impact on neighbors' efficiency})
$$

Example:

- Running node A enables 3 other nodes → high score

4.5 Decay (D(v))

$$
D(v) = \text{CPU time used recently}
$$

Purpose:

- Prevent starvation of others

---

5. Algorithm Workflow

Step 1: Graph Update

- Track:
  - IPC calls
  - shared memory
  - file access
- Update edges dynamically

Step 2: Metric Computation

For each node:

- Compute (R, C, L, E, D)

Step 3: Score Evaluation

- Compute (S(v)) for all runnable nodes

Step 4: Selection

```
fn select_next(nodes: &[Node]) -> Option<&Node> {
    nodes.iter().max_by(|a, b| a.score.partial_cmp(&b.score).unwrap())
}
```

Step 5: Execution

- Dispatch selected node to CPU

Step 6: Feedback Loop

- Update:
  - decay
  - resource field
  - topology weights

---

6. Data Structures (Rust-Oriented)

6.1 Node Structure

```
pub struct Node {
    pub id: usize,
    pub connections: Vec<Edge>,
    pub resource_demand: f64,
    pub allocated: f64,
    pub locality_score: f64,
    pub emergence_score: f64,
    pub cpu_time: f64,
}
```

6.2 Edge Structure

```
pub struct Edge {
    pub target: usize,
    pub weight: f64,
```

```
}
```

6.3 Scheduler Core

```
pub struct Scheduler {
    pub nodes: Vec<Node>,
}

impl Scheduler {
    pub fn compute_score(node: &Node) -> f64 {
        let r = node.resource_demand / (node.allocated + 1.0);
        let c: f64 = node.connections.iter().map(|e| e.weight).sum();
        let l = node.locality_score;
        let e = node.emergence_score;
        let d = node.cpu_time;

        1.0*r + 0.8*c + 0.6*l + 0.7*e - 0.5*d
    }

    pub fn select_next(&self) -> Option<&Node> {
        self.nodes.iter().max_by(|a, b| {
            Self::compute_score(a)
                .partial_cmp(&Self::compute_score(b))
                .unwrap()
        })
    }
}
```

---

7. Emergent Optimization Mechanisms

7.1 Cluster Formation

- Nodes with strong connections form **execution clusters**
- Scheduler prioritizes clusters over isolated nodes

7.2 Load Diffusion

- Overloaded nodes transfer work to neighbors
- Modeled as:
  [
  \text{load flow} \propto \text{gradient difference}
  ]

7.3 Self-Rewiring Graph

- Weak edges decay
- Strong interactions strengthen edges

---

8. Complexity Analysis

Component	Complexity
Score computation	O(n + e)
Selection	O(n)
Graph updates	O(e)

Optimizations:

- Use **priority queues**
- Incremental updates (don’t recompute everything)

---

9. Kernel Integration Strategy

In Linux kernel

Map concepts:

TBS Concept	Linux Equivalent
Node	task_struct
Edge	IPC / shared memory
Score	scheduler priority
Resource field	cgroups

Implementation options:

- Custom scheduler class
- eBPF-based prototype
- Rust kernel module

---

10. Key Innovations vs Traditional Schedulers

Traditional	TBS
Time slicing	Score-based selection
Static priority	Dynamic emergent priority
Isolated processes	Relational graph
Local optimization	Global optimization

---

11. Tuning Strategy

Start with:

```
α = 1.0 (resource urgency)
β = 0.8 (connectivity)
γ = 0.6 (locality)
δ = 0.7 (emergence)
ε = 0.5 (decay)
```

Then:

- Adjust based on workload:
  - High IO → increase locality
  - Distributed → increase connectivity

---

12. Minimal Viable Prototype Plan

1. Implement graph scheduler in Rust (user space)
2. Simulate:
   - 100–1000 nodes
   - random + real workloads
3. Compare against:
   - round-robin
   - priority scheduler

Metrics:

- latency
- throughput
- energy usage

Final Insight

Your scheduler is not just an algorithm—it's a **shift in computational ontology**:

From *"Which process runs next?"*
to *"Which relationship best advances the whole system right now?"*

Below is the code for the Synergetic Scheduler Rust Simulation for the Synergetic Operating system

You can check out the code on github at ➔
https://github.com/solomonyaw/Synergetics-Operating-System/blob/master/synergetic_scheduler_rust_simulation.rs

```
// Synergetic Topology-Based Scheduler Simulation with Visualization

// Run with: cargo run

// Requires: cargo add petgraph rand plotters


use petgraph::graph::{Graph, NodeIndex};

use rand::Rng;

use plotters::prelude::*;


#[derive(Clone, Debug)]

struct NodeData {

    resource_demand: f64,

    allocated: f64,

    cpu_time: f64,
```

```
}

fn compute_score(node: &NodeData, connectivity: f64) -> f64 {
    let r = node.resource_demand / (node.allocated + 1.0);
    let c = connectivity;
    let l = rand::random::<f64>(); // simulated locality
    let e = rand::random::<f64>(); // simulated emergence
    let d = node.cpu_time;

    1.0 * r + 0.8 * c + 0.6 * l + 0.7 * e - 0.5 * d
}

fn main() {
    let mut graph: Graph<NodeData, f64> = Graph::new();
    let mut rng = rand::thread_rng();

    // Create nodes
    let mut nodes = vec![];
    for _ in 0..20 {
        let n = graph.add_node(NodeData {
            resource_demand: rng.gen_range(1.0..10.0),
            allocated: rng.gen_range(1.0..5.0),
            cpu_time: 0.0,
```

```
    });
    nodes.push(n);
}

// Create random connections
for i in 0..nodes.len() {
    for j in i+1..nodes.len() {
        if rng.gen_bool(0.3) {
            graph.add_edge(nodes[i], nodes[j], rng.gen_range(0.5..2.0));
        }
    }
}

let mut history = vec![];

// Simulation loop
for step in 0..100 {
    let mut best_node = None;
    let mut best_score = f64::MIN;

    for node in graph.node_indices() {
        let data = &graph[node];
        let connectivity: f64 = graph.edges(node).map(|e| *e.weight()).sum();
```

```
        let score = compute_score(data, connectivity);

        if score > best_score {
            best_score = score;
            best_node = Some(node);
        }
    }

    // Execute selected node
    if let Some(n) = best_node {
        let node = graph.node_weight_mut(n).unwrap();
        node.cpu_time += 1.0;
        node.allocated += 0.5;
    }

    history.push(best_score);
    println!("Step {}: Best Score = {}", step, best_score);
}

// Visualization
let root = BitMapBackend::new("output.png", (800, 600)).into_drawing_area();
root.fill(&WHITE).unwrap();
```

```
    let mut chart = ChartBuilder::on(&root)
        .caption("Scheduler Score Over Time", ("sans-serif", 30))
        .margin(20)
        .x_label_area_size(30)
        .y_label_area_size(30)
        .build_cartesian_2d(0..100, 0f64..20f64)
        .unwrap();

    chart.configure_mesh().draw().unwrap();

    chart
        .draw_series(LineSeries::new(
            history.iter().enumerate().map(|(x, y)| (x, *y)),
            &RED,
        ))
        .unwrap();

    println!("Visualization saved to output.png");
}
```